

Project Report

Multi-Digit Binary Coded Decimal Arithmetic Units

Reece M. Dobro

Abstract

The process of designing and testing a single-digit and 4-digit Binary Coded Decimal (BCD) adders with carry-in and carry-out using the Verilog hardware description language within AMD Vivado is presented in this report. Two simulations were created to evaluate and examine each module. The results of the simulations were analyzed using waveform outputs and TCL console logs, which confirmed that the outputs of the module matched the expected values. In addition, the RTL Schematic from AMD Vivado was examined and compared to the digital design of the modules. Overall, the simulation results demonstrate that the design operates correctly and satisfies the functional requirements specified by the design specifications.

1 Introduction, Background, and Theory

The main objective of this project is to design, program, and test a 1-digit Binary Coded Decimal (BCD) adder and a 4-digit BCD adder. Both of these adders will have a carry input and carry output. A binary-coded decimal value is a 4-digit binary number that can express values from 0 to 9. The value of a BCD digit is directly encoded in binary. For example, the number 7_{10} in BCD is written as 0111_2 . As the max value of base 10 decimal is 9, any value greater than 9 needs to be adjusted, or else it will be invalid (1010_2 to 1111_2). This correction occurs through the addition of 6_{10} and a carry-out value of 1. 6_{10} is added as there are 6 invalid values for BCD in a 4-bit binary number (10_{10} to 15_{10}). The 1-digit BCD adder described in this report accepts two BCD operands and a 1-bit carry-in value as per design specifications. This is performed by adding the operands and carry-in value in binary, then converting the sum to a valid BCD sum with a carry-out value. For this single-digit BCD adder, it converts numbers when the initial binary addition produces a carry-out or if the intermediate 4-bit sum falls within the invalid range. The 4-digit BCD adder has four instances of the 1-digit BCD adders that are ripple-carry to create a 4-digit BCD adder. This is done by connecting the carry out of each individual stage to the carry-in of the next BCD adder stage. The operand inputs for each stage are mapped to corresponding 4-bit segments of the 16-bit input vectors.

1.1 Inputs and Outputs

The inputs, outputs, and expected behavior were provided by the design specifications for this system. The inputs and outputs for both modules are shown in Tables 1 and 2. As the 1-digit BCD adder is 4 times smaller than the 4-digit BCD adder, its inputs and outputs are as well.

Signal	Direction	Size	Description
A	Input	4	First BCD Operand
B	Input	4	Second BCD Operand
Cin	Input	1	Carry-in value
S	Output	4	BCD sum output
Cout	Output	1	Carry-out value

Table 1: 1-digit BCD Adder Port Signal and Function Table

Signal	Direction	Size	Description
A	Input	16	First BCD Operand
B	Input	16	Second BCD Operand
Cin	Input	1	Carry-in value
S	Output	16	BCD sum output
Cout	Output	4	Carry-out value

Table 2: 4-digit BCD Adder Port Signal and Function Table

1.2 Design of 1-digit BCD Adder

Figure 1 shows the structural schematic of the 1-digit BCD adder. The module name for the 1-digit BCD adder is *BCDadd_1d*, as per design specifications. As shown in Figure 1, the design consists of two binary adders, two comparators, and a 2-to-1 multiplexer. The process of the design consists of first adding the two operands and carry-in value through binary, checking to see if the sum of that addition is an overflown value, and adjusting accordingly if so. As mentioned previously, if a BCD digit overflows, it will need to have 6 added to it and have a carry-out value of 1. Additionally, if the intermediate result exceeds 9 (1001₂), even without producing a carry, 6 must still be added to obtain a valid BCD digit.

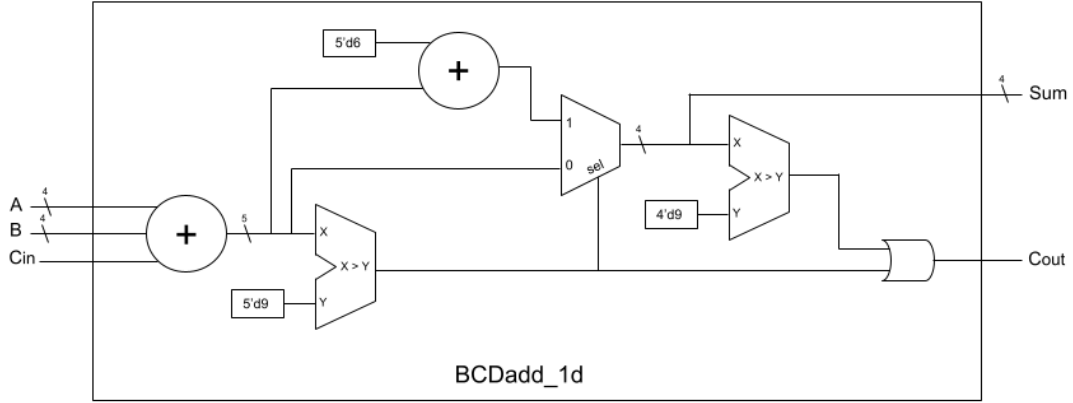


Figure 1: Structural Schematic of the 1-digit BCD adder

1.3 Design of 4-digit BCD Adder

Figure 2 shows the structural schematic of the 4-digit BCD adder. The module name for the 4-digit BCD adder is *BCDadd_4d*, as per design specifications. As shown in Figure 2, the design consists of 4 instances of the 1-digit BCD adder, *BCDadd_1d*. The carries of the 4 instances are ripple carried, with the initial carry being provided by the 1-bit input, *carry_in*. The design of this module is similar to a ripple carry adder (RCA), with full adder instances being used, and 1-digit BCD adders are. In addition, each instance of the 1-digit BCD adders inputs and outputs four bits, unlike the full adders for an RCA design that have one bit.

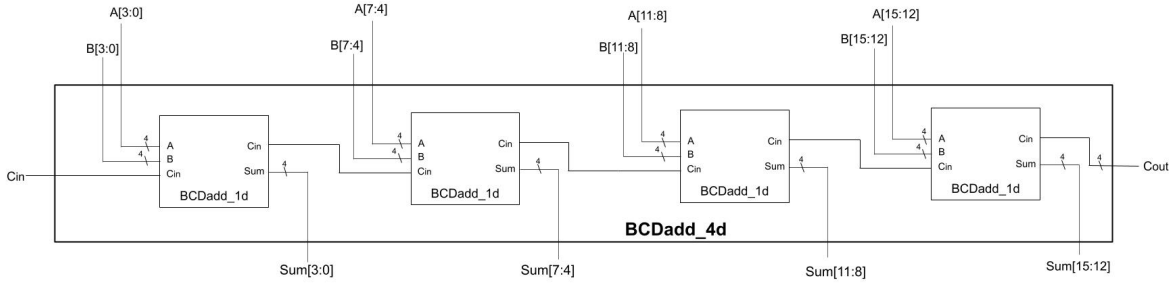


Figure 2: Structural Schematic of the 4-digit BCD adder

1.4 Verilog Implementation

The Verilog implementation of the 1-digit BCD adder, *BCDadd_1d*, and the 4-digit BCD adder, *BCDadd_4d*, was done through dataflow modeling. While the modules are written at the dataflow level, as they perform combinational logic, they could have been written at the behavioral level. Figure 3 shows the source code of the module, *BCDadd_1d*. The source code of the module is based on the architectural diagram of the 1-digit BCD adder, Figure 1.

```

module BCDadd_1d(
    input [3:0] A,
    input [3:0] B,
    input Cin,
    output [3:0] Sum,
    output Cout
);

    wire [4:0] preSum;
    wire comp1;
    wire comp2;
    wire [3:0] convA;
    assign preSum=A+B+Cin;
    assign comp1=preSum>5'd9;
    assign convA=preSum+5'd6;
    assign Sum=comp1?convA:preSum;
    assign comp2=Sum>4'd8;
    assign Cout=comp1|comp2;
endmodule

```

Figure 3: Source Code of Module *BCDadd_1d*

Figure 4 shows the source code for the module, *BCDadd_4d*, the 4-digit BCD adder. Its source code is based on the architectural diagram of the 4-digit BCD adder, Figure 2. As design specifications required that all code be written at the behavioral level, the size of the carry out signal, *Cout* is 4 bits. This was achieved by assigning the LSB of *Cout* to the carry-out wire of the final instance of the 1-digit BCD adder, *u4*. The internal net names were chosen based on the function they execute. For example, the 4-bit wire *Cinter* was given its name as it serves the purpose of providing an intermediate connection between the last carry-out and the output, *Cout*.

```

module BCDadd_4d(
    input [15:0] A,
    input [15:0] B,
    input Cin,
    output [15:0] Sum,
    output [3:0] Cout
);
    wire [3:0] Cinter;
    BCDadd_1d u1(A[3:0],B[3:0],Cin,Sum[3:0],Cinter[0]);
    BCDadd_1d u2(A[7:4],B[7:4],Cinter[0],Sum[7:4],Cinter[1]);
    BCDadd_1d u3(A[11:8],B[11:8],Cinter[1],Sum[11:8],Cinter[2]);
    BCDadd_1d u4(A[15:12],B[15:12],Cinter[2],Sum[15:12],Cinter[3]);
    assign Cout=Cinter[3];
endmodule

```

Figure 4: Source Code of Module *BCDadd_4d*

2 Simulation, Results and Analysis

To verify the accuracy of the *BCDadd_1d* module and the *BCDadd_4d* module, four distinct simulation scenarios will be executed. Each simulation will consist of two operands and a carry in being entered as inputs, with the outputs of the module under test and its behavior being analyzed to determine if it is successful. The test cases chosen for the four simulations are shown in Table 3 and Table 4.

Test Case	Inputs			Expected Outputs	
	A [3:0]	B [3:0]	Cin	Sum [3:0]	Cout
Boundary	4'b1001	4'b0001	0	4'b0000	1
Max Value	4'b1001	4'b1001	1	4'b1001	1

Table 3: Single-Digit BCD Adder *BCDadd_1d* Test Vectors

The first test case listed in Table 3 is a boundary test case. Its primary purpose is to verify that the correction logic triggers properly when the sum exceeds the maximum BCD digit value of 9. In addition, it tests the independence of the two operands, with $A = 9$ and $B = 1$. By using asymmetric operands, this test case ensures that the module is not relying on having the same value for both operands. This confirms that both the A and B ports are independently routed to the internal adder logic.

The second test case listed in Table 3 is a maximum value test case. Its primary purpose is to verify that the BCD adder can handle the worst-case scenario for carry propagation and the largest possible sum. In addition to this, this test case also sets the carry-in input, *Cin*, high, ensuring that input carry propagation works as expected.

Test Case	Inputs			Expected Outputs	
	A [15:0]	B [15:0]	Cin	Sum [15:0]	Cout [3:0]
Ripple Carry	16'h0999	16'h0000	1	16'h1000	4'h0
Global Carry	16'h8000	16'h2000	0	16'h0000	4'h1

Table 4: 4-Digit BCD Adder *BCDadd_4d* Test Vectors

The first test case listed in Table 4 is a ripple carry test case. Its primary purpose is to verify that carries can propagate through the 4 instances of the 1-digit BCD adder. This is performed by forcing a carry to be generated in the ones digit (*Sum[3:0]*), which then ripples into the Tens digit (*Sum[7:4]*), which ripples into the Hundreds digit (*Sum[11:8]*), finally terminating in the Thousands digit (*Sum[15:12]*). By having the input, *Cin*, driven high, it ensures that the initial carry-in logic performs as expected.

The second test case listed in Table 4 is a global carry test case. Its primary purpose is to verify that the 4-digit BCD adder correctly handles an overflow of the MSB and properly formats the final carry-out. In addition to this, by setting the lower twelve bits of the operands (*A[11:0]* and *B[11:0]*) to zero, this test case ensures that the global carry-out is generated only by the MSB and not by a ripple effect from lower stages. By having the carry-out be 1, it also ensures the final carry-out logic behaves appropriately.

To execute the four test cases, four testbenches were created in Verilog. Each simulation will output its critical information through both the TCL Console and the waveform. The waveforms output ensures that ports are mapped according to design specifications and can help to debug any issues discovered during testing. The TCL Console displays outputs in a text-based format that allows for simpler analysis of the ports. The inputs for each simulation match those given in Tables 3 and 4. The length of each simulation is 10 ns, with the units of time being 1 ns. As both of the modules under test consist of only combinational logic, no clock is needed for any simulations.

2.1 Simulation Results

Tables 5 and 6 contain the results of the four simulations compared to the expected results. The TCL Console output from the simulations was compiled into the tables to show the actual outputs and allow for direct comparison to the expected outputs.

Test Case	Inputs			Expected Outputs		Actual Outputs	
	A [3:0]	B [3:0]	Cin	Sum	Cout	Sum	Cout
Boundary	4'b1001	4'b0001	0	4'b0000	1	4'b0000	1
Max Value	4'b1001	4'b1001	1	4'b1001	1	4'b1001	1

Table 5: Single-Digit BCD Adder *BCDadd_1d* Test Vectors

Test Case	Inputs			Expected Outputs		Actual Outputs	
	A [15:0]	B [15:0]	Cin	Sum	Cout	Sum	Cout
Ripple Carry	16'h0999	16'h0000	1	16'h1000	4'h0	16'h1000	4'h0
Global Carry	16'h8000	16'h2000	0	16'h0000	4'h1	16'h0000	4'h1

Table 6: 4-Digit BCD Adder *BCDadd_4d* Test Vectors

The waveforms output of the Boundary test case is shown in Figure 5. The radix for all values is binary to match the formatting of Table 5. As shown by the figure, the outputs of the module, $Sum = 4'b0000$ and $Cout = 1'b1$, match the expected behavior given by the design specifications. As the operands were computed correctly, the simulation confirms that the internal correction logic successfully identifies a sum greater than 9 and applies the required +6 offset.

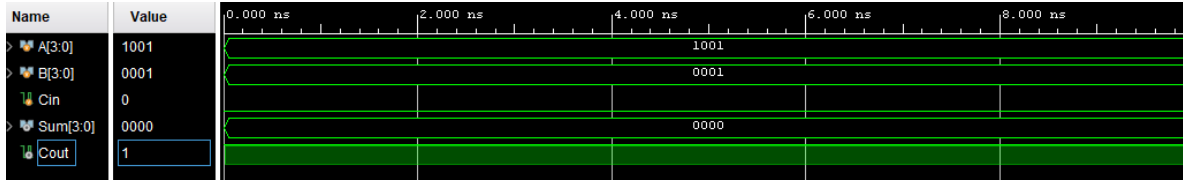


Figure 5: Waveforms Output of Boundary Test Case

The waveforms output of the Max Value test case is shown in Figure 6. The radix for all values is binary to match the formatting of Table 5. As shown by the figure, the outputs of the module, $Sum = 4'b1001$ and $Cout = 1'b1$, match another behavior set by design specifications. As the operands were computed correctly, the module remains operational at its highest possible sum. By producing a result of 19 (represented as a binary 9 with a high carry-out), the simulation proves that the correction logic effectively handles cases where the initial binary sum significantly exceeds the BCD threshold while generating the proper carry-out value.

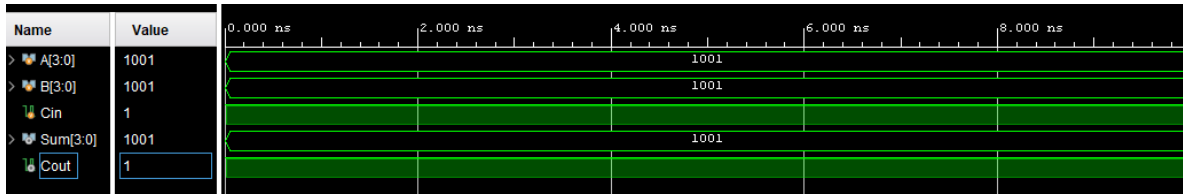


Figure 6: Waveforms Output of Max Value Test Case

The waveforms output of the Ripple Carry test case is shown in Figure 7. The radix for all values is hexadecimal to match the formatting of Table 6. As shown by the figure, the outputs of the module, $Sum = 16'h1000$ and $Cout = 1'b0$, match the appropriate behavior set in the architectural diagram and design specifications. As the operands were computed correctly, the simulation confirms that the carry-out from each *BCDadd_1d* instance successfully serves as the carry-in for the next digit. This validates the ripple-carry design of the module when handling BCD values.

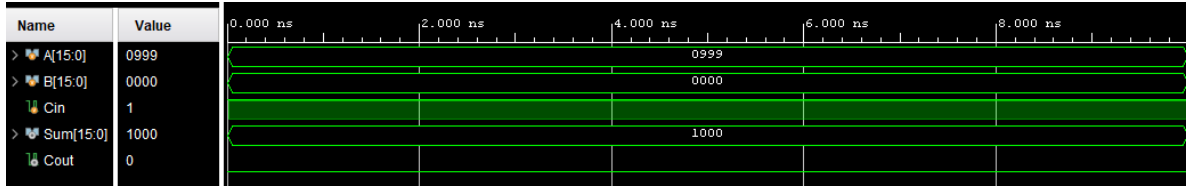


Figure 7: Waveforms Output of Ripple Carry Test Case

The waveforms output of the Global Carry test case is shown in Figure 8. The radix for all values is hexadecimal to match the formatting of Table 6. Unlike Figure 7, the 16-bit output *Sum* has all bits held low. As only the LSB of the 4-bit output, *Cout* is held high. This shows the success of the module for this test case. Since the sum of 8000 and 2000 equals 10000, which exceeds the four-digit BCD maximum, the simulation confirms that the fifth digit is correctly captured by the overflow port. This validates that the ripple carry design can effectively pass the final carry-out bit to the top-level output and ensures that the correct data are still given, even when the total sum exceeds 9999.

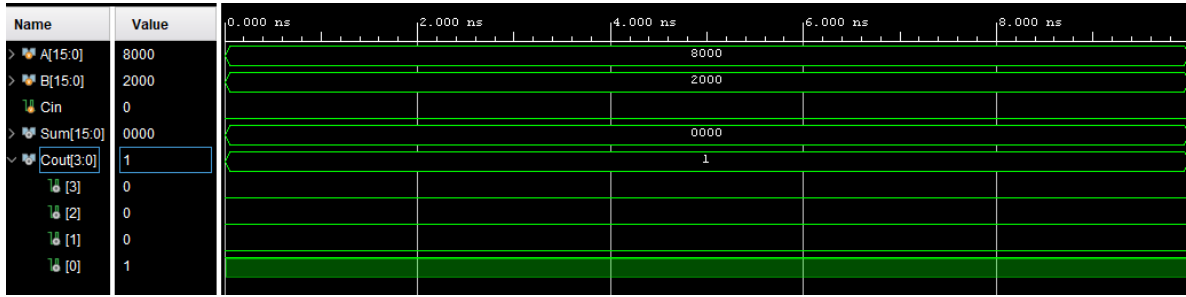


Figure 8: Waveforms Output of Global Carry Test Case

As the output is given at the start of all of the simulations and remains constant, the modules and design are shown to be executing only combinational logic.

A potential limitation in the verification process is that only a limited number of test cases were utilized. For the single-digit *BCDadd_1d* module, exhaustive testing is simple, as there are only $2^4 \times 2^4 \times 2^1 = 512$ total input combinations. However, for the 4-digit *BCDadd_4d* module, the complexity increases significantly. With two 16-bit inputs, there are $2^{16} \times 2^{16} \times 2^1 = 2^{37} = 137,438,953,472$ possible input combinations. Even when restricting inputs to valid BCD digits (0-9), there are $10^4 \times 10^4 \times 2 = 200,000,000$ valid combinations. While the simulation scenarios cover the primary operations, they represent only a fraction of the total input space. However, it is impractical to simulate every possible 16-bit sequence due to the time required for manual verification and documentation. This is why representative test cases, specifically boundary cases and multi-digit ripple carry sequences, were selected to confirm the design's success under a variety of logical conditions.

Both modules are deemed successful as the output waveforms and TCL binary logs match the expected behavior set by the design specifications and match the expected outputs.

2.2 RTL Schematic Results and Comparison

Figures 9 and 10 show the schematic of the RTL Analysis from AMD Vivado for both modules, with Figure 9 showing the 1-digit BCD adder and Figure 10 showing the 4-digit BCD adder. These figures provide a view of the synthesized design as interpreted by the AMD Vivado. It shows the Verilog program translated into basic hardware components such as multiplexers and logic gates. For Figure 10, the instances featured in the design aren't expanded as the expanded design is shown in Figure 9.

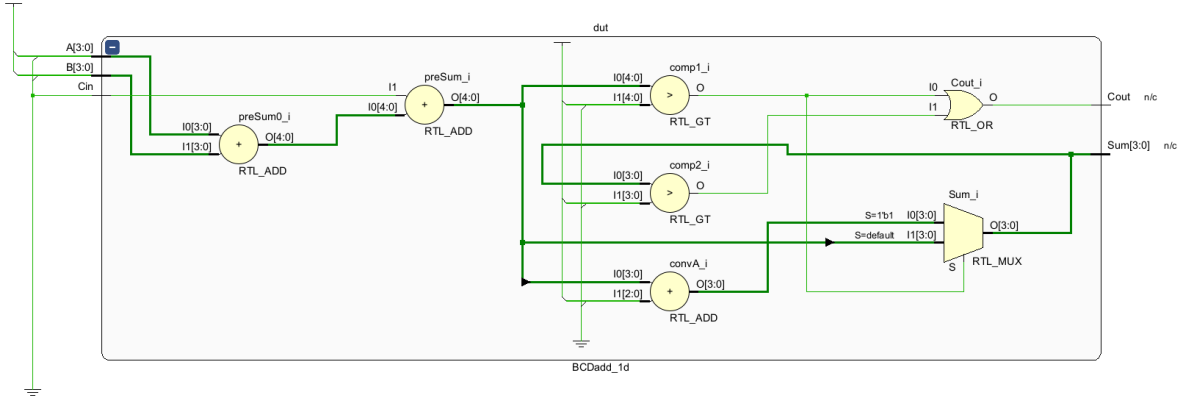


Figure 9: RTL Schematic result for the module, *BCDadd_1d*

The schematic of the 1-digit BCD adder, Figure 9, matches closely with the expected results from the architectural diagram created for the module, Figure 1. The only difference between the two diagrams is the intermediate adder used to calculate the sum of the two BCD digits, *A* and *B*, and the carry-in value. The adder design in figure 1 is a standard adder with three inputs, while the adder design in figure 9 is a 2-stage adder. In the RTL schematic, the sum of *A* and *B* is calculated first; this sum then feeds into a second adder to incorporate the carry-in value. All other elements of the design, including the 2 comparators, additional adder, 2 to 1 multiplexer, and OR gate, remain the same, along with the connections between these elements. Due to the close similarity between these designs, it can be concluded that the Verilog implementation of the architectural diagram was successfully done.

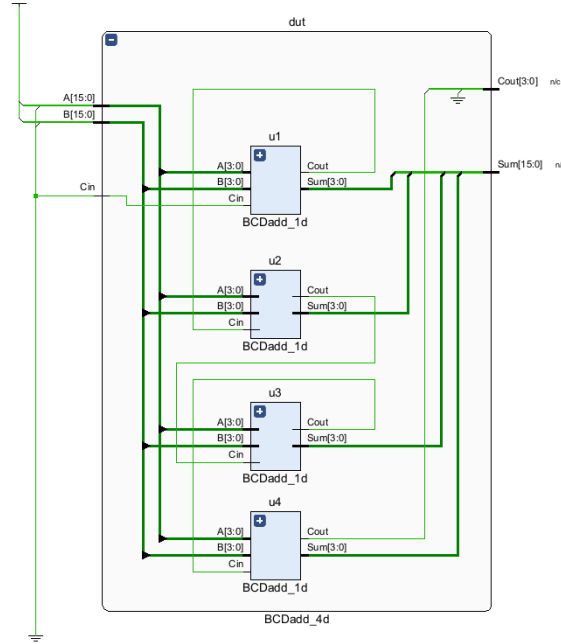


Figure 10: RTL Schematic result for the module, *BCDadd_4d*

The schematic of the 4-digit BCD adder, Figure 10, matches exactly with the expected results from the digital diagram created for the module, Figure 2. This includes the 4 instances of *BCDadd_1d* and the internal nets and ports. The "ripple carry" implementation is also shown in the figure with the carry out of the first instance, *u1*, connecting to the carry in of *u2*. Due to the close similarity between these designs, it can be concluded that the Verilog implementation of the architectural diagram was successfully done.

3 Conclusions

In this project, the correct design, implementation, and verification of a 1-digit BCD adder and a 4-digit BCD adder with carry-in and carry-out functionality was achieved. The simulation results confirmed that both modules work correctly and were able to convert invalid BCD values into carries. Simulation results also confirmed that the modules can handle edge cases, such as maximum values. The implementation was shown to be successful, with the RTL schematic matching the digital design. Future work could include resource analysis to identify potential hardware modifications that would improve overall efficiency. Overall, this project demonstrates how Verilog can be used to implement and verify arithmetic modules digitally, producing modular designs without requiring physical components or additional hardware.

This report was compiled using L^AT_EX .